CSC 212: Data Structures and Abstractions

13: Binary Search Trees (part 1)

Prof. Marco Alvarez

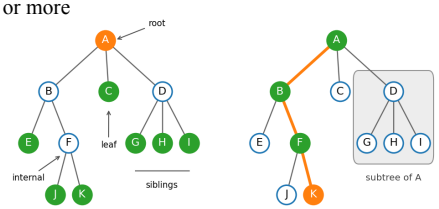
Department of Computer Science and Statistics
University of Rhode Island

Fall 2026



Trees

- A **tree** is a set of nodes in a **parent-child** relation
  - a nonempty tree has one **root**, the only node with no parent
  - every other node has exactly one parent
- A **leaf** (external node) has no children; an **internal node** has one or more

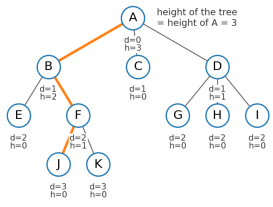


Ancestors, descendants, subtrees

- *u* is an **ancestor** of *v* if *u* = *v*, or *u* is an ancestor of the parent of *v*
  - a recursive definition: it follows parents up to the root
  - *proper ancestor* excludes *u* = *v*
- *v* is a **descendant** of *u* if *u* is an ancestor of *v*
  - *proper descendant* excludes *u* = *v*
- The **subtree** rooted at *v:* all its descendants
- An **edge** is a parent-child pair; a **path** is a sequence of nodes such that each consecutive pair forms an edge

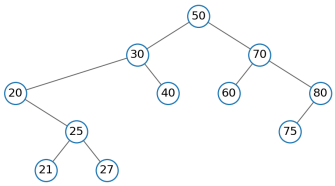
Depth and height

- The **length of a path** is its number of edges
- The **depth** of *v* is the length of the path from the root down to node *v*
- The **height** *v* is the length of the longest path from *v* down to a leaf
  - every leaf has height 0
- The **height of the tree** is the height of the root



Q13-1

- In this tree, what are the depth of 25 and the height of 30?



A. 2 and 2
B. 3 and 3
C. 3 and 4
D. 4 and 3
E. None of the above

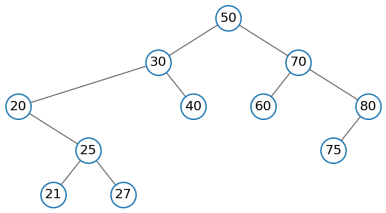
Answer

- B) 3 and 3

  - depth is measured from the top, height from the bottom
  - the height of the whole tree is 4: from 50 down to 21

Q13-2

- Label every node of this tree with its depth and its height
- What is the height and the depth of the tree?
- Which nodes are leaves and which are internal nodes?



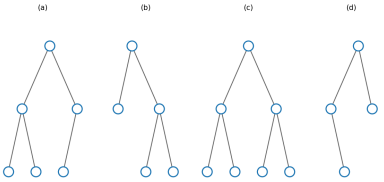
k-ary trees

- ***k*-ary tree**: every internal node has at most *k* children
- **Full tree**: every internal node has **exactly** *k* children
- **Complete tree**: levels 0 to *h-1* are full, and level *h* is filled **from the left**
  - a binary heap is a complete binary tree, stored in an array
- **Perfect tree**: every internal node has *k* children, and all leaves are at the same depth
  - a perfect binary tree of height *h* has $1 + 2 + 4 + \cdots + 2^h$ nodes

$$1 + 2 + 4 + \ldots + 2^h = \sum_{i=0}^{h} 2^i = 2^{h+1} - 1$$

Q13-3

- Give each binary tree its most specific name: perfect, complete, full, or none of these



A. (a) complete, (b) full, (c) perfect, (d) none
B. (a) full, (b) complete, (c) perfect, (d) complete
C. (a) perfect, (b) full, (c) complete, (d) full
D. (a) complete, (b) complete, (c) full, (d) none
E. None of the above

Answer

- A) (a) complete, (b) full, (c) perfect, (d) none
  - (a): the last level is filled from the left, but one node has a single child: complete, not full
  - (b): every internal node has 2 children, but the last level is not filled from the left: full, not complete
  - (c): perfect, so it is also full and complete
  - (d): one node has a single child, and it is a right child: neither
  - perfect implies full and complete; neither of those implies the other

Binary trees

- **binary tree**: every node has at most two children, a **left child** and a **right child**
  - the subtrees rooted at them are the left subtree and right subtree
  - k-ary tree, where $k = 2$
- A recursive definition, a **binary tree** is either:
  - empty, or
  - a node with a left binary tree and a right binary tree

```
struct Node {
    int key;
    Node* left; // root of the left subtree, or nullptr
    Node* right; // root of the right subtree, or nullptr
    Node(int k) : key(k), left(nullptr), right(nullptr) {}
};
```

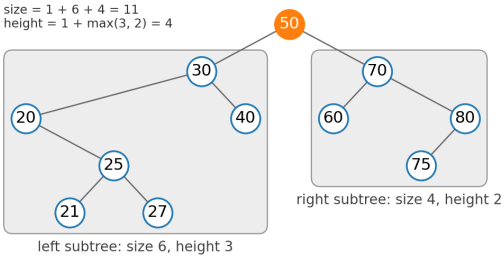

Recursion follows the shape

- The definition is recursive
  - **methods** are too: solve the problem at one node, and let the calls handle the subtrees

```
size_t size(Node* p) {
    if (p == nullptr) return 0; // empty tree
    return 1 + size(p->left) + size(p->right);
}


int height(Node* p) {
    if (p == nullptr) return -1; // empty tree
    return 1 + std::max(height(p->left), height(p->right));
}
```

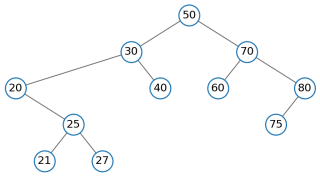
a leaf has height 1 + max(-1, -1) = 0



Each node is visited once: both methods are $\Theta(n)$

Q13-4

- A student writes `height` with `return 0;` for the empty tree, and keeps the rest. What does it return for the tree below, and for a tree with one node?



A. 4 and 0
B. 5 and 1
C. 4 and 1
D. 3 and 0
E. None of the above

Answer

- B) 5 and 1

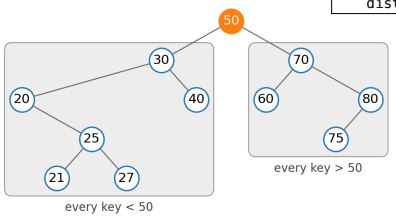
- ✓ every result shifts by one: the function now counts nodes on the longest path, not edges
- ✓ the base case decides what the whole recursion means

Binary search trees

Binary search tree

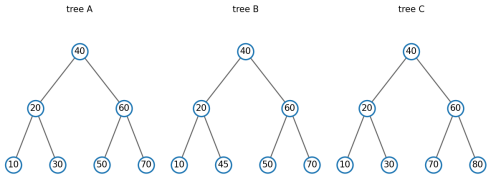
- A **binary search tree (BST)** is a binary tree in which every node x satisfies the BST property (symmetric order)
  - every key in the left subtree of x is $< key(x)$
  - every key in the right subtree of x, is $> key(x)$

assume that the keys of a BST are pairwise distinct



Q13-5

- Which of these trees are binary search trees?

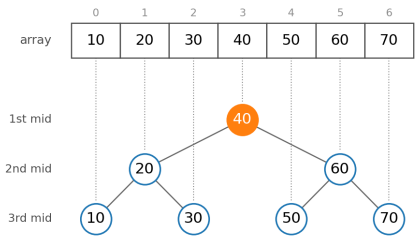


A. A only
B. A and B
C. A and C
D. All three
E. None of the above

Answer

- A) A only
  - in B) 45 is in the wrong place
  - in C) 70 is in the wrong place

Binary search and a binary search tree



Binary search walks one path down this tree, from the root.

The class

```
class BST {
    private:
        Node* m_root; // nullptr when the tree is empty
        size_t m_size;

        static bool search_helper(Node* p, int key);
        Node* insert_helper(Node* p, int key);
        static void clear_helper(Node* p);

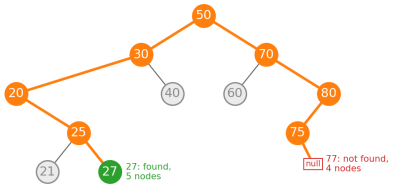
    public:
        BST() : m_root(nullptr), m_size(0) {}
        ~BST() { clear_helper(m_root); }

        bool search(int key) const;
        void insert(int key);
        size_t size() const { return m_size; }
        bool empty() const { return m_size == 0; }
};
```

keys are `int` to keep it short; a template version could also be implemented

Search

- Start at the root, and at each node `p`
  - `key == p->key`: found
  - `key < p->key`: it can only be in the left subtree, so go left
  - `key > p->key`: go right
  - `p == nullptr`: we fell off the tree: not found



`search_helper`

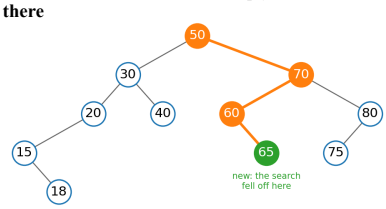
```
bool BST::search(int key) const {
    return search_helper(m_root, key);
}

bool BST::search_helper(Node* p, int key) {
    if (p == nullptr)
        return false; // not in the tree
    if (key < p->key)
        return search_helper(p->left, key);
    if (key > p->key)
        return search_helper(p->right, key);
    return true; // key == p->key
}
```

Time complexity: $O(h)$, where h is the height of the tree

Insert

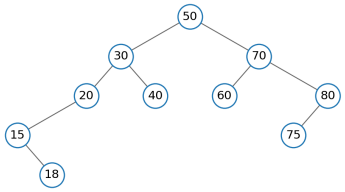
- Search for the key
  - if it is found, there is nothing to do
  - otherwise, the search falls off at an empty link: **insert the new node there**



The new node is always a leaf, and no other node moves
(the insertion only changes one pointer)

Q13-6

- Insert 65, 27, 90, 11, 51, in this order, into the tree below. Which node is the parent of 51?



A. 50
B. 60
C. 65
D. 70
E. None of the above

Answer

- B) 60

[draw the tree and perform the insertions]

insert_helper

```
void BST::insert(int key) {
    m_root = insert_helper(m_root, key);
    ++m_size;
}


Node* BST::insert_helper(Node* p, int key) {
    if (p == nullptr)
        return new Node(key);
    if (key < p->key)
        p->left = insert_helper(p->left, key);
    else if (key > p->key)
        p->right = insert_helper(p->right, key);
    return p;
}
```

The helper returns the root of the updated subtree, and the caller stores it back.
Time complexity: O(h), where h is the height of the tree.

Q13-7

- With the code on the previous slide, a program inserts 50, 30, 50, 70, 30 into an empty BST. What does `size()` return?

```
Node* BST::insert_helper(Node* p, int key) {
    if (p == nullptr)
        return new Node(key);
    if (key < p->key)
        p->left = insert_helper(p->left, key);
    else if (key > p->key)
        p->right = insert_helper(p->right, key);
    return p;
}
```

A. 2
B. 3
C. 4
D. 5
E. None of the above

Answer

- D) 5
  - however, the tree has only 3 nodes
    - `insert` counts calls, not new nodes: the duplicates 50 and 30 are not added to the tree, yet `m_size` grows
    - **fix:** count where the node is created, the only place that knows

```
void BST::insert(int key) {
    m_root = insert_helper(m_root, key);
}

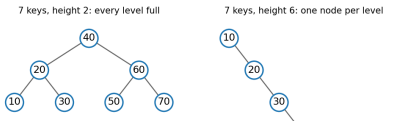
Node* BST::insert_helper(Node* p, int key) {
    if (p == nullptr) {
        ++m_size; // only a new node changes the size
        return new Node(key);
    }
    if (key < p->key)
        p->left = insert_helper(p->left, key);
    else if (key > p->key)
        p->right = insert_helper(p->right, key);
    return p;
}
```

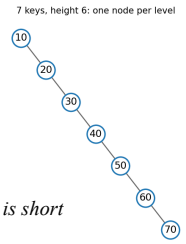

Repeated keys

- So far, keys are **pairwise distinct**. When a key is inserted twice, choose a policy:
  - **ignore the duplicate**: a set
    - our `insert_helper`, `std::set`
  - **count it**: store a counter in each node, and add 1 on a duplicate
    - a word-frequency table
  - **update the value**: the node stores a *(key, value)* pair, and the new value replaces the old one
    - a map: `std::map`, Java's `TreeMap`

How tall is a BST?

- Search and insert each walk one path: both cost $\Theta(h)$
  - but h depends on the shape, and the same keys fit many shapes





A search tree is only as fast as it is short

The search problem

- Collections
  - a **set** stores keys and answers three requests
  - a **map** stores a value with each key
  - operations: `search(k)`, `insert(k)`, `remove(k)`

data structure	`search`	`insert`
unsorted vector	Θ(n)	Θ(1) amortized
sorted vector	Θ(log n)	Θ(n)
binary heap	Θ(n)	Θ(log n)
binary search tree	Θ(h)	Θ(h)

The unsorted vector's insert is Θ(1) only if duplicates are allowed; rejecting a duplicate needs a search first. With h between log n and n.

Exercises

- Write `size_t count_leaves(Node* p)`
  - returns the number of leaves in the tree rooted at p
  - hint: the empty tree has 0 leaves; a node with no children is 1 leaf
  - what does it cost?
- Write `bool BST::search(int key)`
  - an iterative method for searching a key in a BST
- Write `void BST::insert(const int& key)`
  - an iterative method for inserting a key into a BST